TASK 4 REPORT

C++ Project — OOP & STL Mini Application

Library Management System | Design and Testing Report

Student Name	Sohail Khan
Internship	C / C++ Programming Internship
Task Number	4

This report explains the task goal, design, implementation plan, sample test cases, complexity, and steps for building and submitting the project on GitHub.

1. Objective

Build a small C++ application using classes, object-oriented programming (OOP), STL containers, and file persistence. This report uses a **Library Management System** example, one of the suggested mini-applications in the internship task.

- Use classes to represent books and the library manager.
- Use STL containers such as vector or map to store records.
- Save and load records from a file so data can persist between runs.
- Provide build instructions, sample data, and basic tests.

2. Application Features

- **Add book:** store a unique book ID, title, author, and availability status.
- **List books:** display all stored records.
- **Search:** find a book by ID or title.
- **Issue book:** mark an available book as issued.
- **Return book:** mark an issued book as available.
- **Persistence:** load records at startup and save changes before exit.

3. OOP Design

Book class

The **Book** class contains private data members such as id, title, author, and isIssued. Public methods or controlled accessors allow the program to display or update book information without exposing all fields directly.

Library class

The **Library** class manages a collection of Book objects, for example `vector<Book>`. Its methods handle adding, searching, issuing, returning, loading, and saving books.

Design benefits

- Encapsulation keeps data and operations together.
- Separate classes make the program easier to test and maintain.
- STL containers reduce the need to manage a custom dynamic array.
- File handling keeps records available after the application closes.

4. Sample User Session

The following is an illustrative interaction. Match the actual prompts and output to the final source code before submitting.

```
===== Library Management System =====\n1. Add book\n2. List books\n3. Search book\n4. Issue book\n5. Return book\n0.
```

For invalid IDs, duplicate IDs, unavailable books, and missing files, show a clear message rather than crashing.

5. File Format and Error Handling

A simple text file such as **books.txt** can store one book per line. Choose a delimiter that will not normally appear in a title, or validate/escape input if using a delimiter-based format.

```
101|C++ Basics|A. Student|0\n102|Data Structures|B. Student|1
```

In this example, 0 means available and 1 means issued. The exact format is a design choice; ensure the load and save functions use the same format. Check whether the file opened successfully and handle malformed records safely.

6. Test Cases

- Add a new book and confirm it appears in the list.
- Try adding a duplicate ID; the program should reject it.
- Search for an existing ID and a missing ID.
- Issue an available book; issuing it again should fail.
- Return an issued book; returning an already available book should show a message.
- Exit and restart the program; saved books and statuses should remain.
- Test an empty or missing data file and verify the program handles it gracefully.

7. Build and Run

Save the main program as **library_management.cpp**. Compile with GCC/G++:

```
g++ -std=c++17 -Wall -Wextra -pedantic library_management.cpp -o library_app\n./library_app
```

Keep **books.txt** in the working directory if the application uses a relative file path. The program may create the file automatically if it does not exist.

8. Suggested GitHub Structure and Conclusion

```
task-4-library-management/\n■■■ library_management.cpp\n■■■ books.txt\n■■■ README.md\n■■■ Makefile\n■■■ screensh
```

This mini-application demonstrates classes, encapsulation, STL containers, menu-driven interaction, and file persistence. Before submission, make sure the repository contains the working source code and sample data, update the README with exact commands, and capture screenshots from the actual running program.